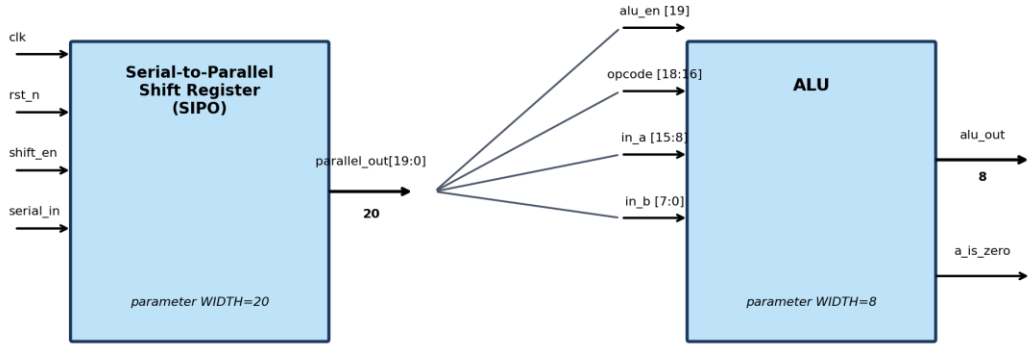
Lab 5-2 Serial-to-Parallel Register Feeding a 3-Operation ALU

Objective: To use Verilog sequential and combinational constructs to build a serial-to-parallel (SIPO) shift register whose 20-bit parallel output configures and drives a parameterized-width ALU.

This lab extends the ALU design from Lab 5-1. Instead of driving in_a, in_b, and opcode directly, these values now arrive serially, one bit at a time, and are assembled by a shift register into a single 20-bit word. That word is then split into fields that feed the ALU. This introduces students to interfacing a sequential block (the shift register) with a combinational block (the ALU), and to the importance of fully specifying case statements in combinational logic.

Block Diagram



Part A — Serial-to-Parallel Shift Register

Design a module named sipo_reg that assembles a 20-bit word from a serial input stream, MSB first.

Port	Width	Description
clk	1	System clock. Sampling/shifting occurs on the rising edge.
rst_n	1	Active-low asynchronous reset. Clears parallel_out to 0.
shift_en	1	Shift enable. When 1, serial_in is shifted in on the next clock edge.
serial_in	1	Serial data input, MSB-first.
parallel_out	20	Parallel output register; holds the last 20 bits received.

Design requirements:

- Parameterize the register width (default WIDTH = 20) so the module can be reused for other widths.
- On each rising edge of clk, if shift_en = 1, shift parallel_out left by one bit and load the incoming serial_in into bit [0] (i.e., data arrives MSB first).
- rst_n is asynchronous and active-low; when asserted (0), parallel_out is cleared to all zeros regardless of clk.
- After 20 shift_en pulses, parallel_out holds one complete 20-bit command word, as described in the bit-field table below.

Bit-Field Assignment

Once fully loaded, parallel_out[19:0] is interpreted as follows:

Signal	Bit Range	Width	Description
alu_en	parallel_out[19]	1	ALU enable (MSB of the shift register output)
opcode	parallel_out[18:16]	3	ALU operation select
in_a	parallel_out[15:8]	8	ALU operand A
in_b	parallel_out[7:0]	8	ALU operand B

Part B — The ALU

Design a module named alu (you may reuse and extend your Lab 5-1 design) that accepts the fields produced by the shift register.

Port	Width	Description
in_a	8	Operand A, taken from parallel_out[15:8]
in_b	8	Operand B, taken from parallel_out[7:0]
opcode	3	Operation select, taken from parallel_out[18:16]
alu_en	1	Enable, taken from parallel_out[19] (the MSB)
alu_out	8	Result output
a_is_zero	1	Asynchronous flag: 1 when in_a == 0, else 0

Specifications:

- Parameterize the ALU operand/result width (default WIDTH = 8) and assign it a default value.
- a_is_zero is combinational: it is 1 whenever in_a = 0, and 0 otherwise, regardless of alu_en or opcode.
- When alu_en = 0, the ALU is disabled: force alu_out = 0.
- When alu_en = 1, alu_out is selected by opcode according to the table below. You may choose the specific arithmetic/logic functions for opcodes 000–010 — ADD, SUB, and AND are suggested, but any three distinct operations are acceptable.

Opcode [2:0]	Instruction	Operation	alu_out
000	ADD	Addition	in_a + in_b
001	SUB	Subtraction	in_a - in_b
010	AND	Bitwise AND	in_a & in_b
011	XOR	Bitwise XOR	in_a ^ in_b
100	OR	Bitwise OR	in_a | in_b
101	OUT A	Input A is Output	in_a
100 - 111	(default)	No operation	alu_out = 8'b0

End of Lab